

NISQ 时期下针对对角线酉矩阵的近似综合算法研究

摘要 量子线路综合问题是含噪声中等规模量子计算时代 (noisy intermediate-scale quantum (NISQ) era) 连接量子算法设计与量子设备设计的桥梁。在这之中，对角线酉矩阵综合问题由于其基础性及泛用性在量子线路综合问题中有至关重要的作用。同时现有的实验结果发现，在实施量子算法时适当降低算法准确性进而降低量子资源消耗能在一定程度上增加算法总的保真度。基于此，我们提出了一种新的量子线路合成算法，该算法能够针对给定的量子资源上限或误差上限设计对应的对角线酉矩阵实现线路。在普通笔记本电脑上，我们的算法可以综合任意比特数为 12 的对角线酉矩阵。当比特数为 8 时，且允许对角线酉矩阵误差至多为 9.73% 时，我们的算法可以节省 21.1% 的 $CNOT$ 门。当比特数为 10 时，且允许对角线酉矩阵误差至多为 11.96% 时，我们的算法可以节省 19.94% 的 $CNOT$ 门。如果将误差放松到 18.98% 时，我们的算法可以至多节省 39.39% 的 $CNOT$ 门。此外，同时将该算法封装成为软件包，这样的工具可以促进使用人员的算法探索，便于没有线路综合基础的研究人员使用该算法。

关键词 量子计算；量子线路；状态制备；酉合成；蒙特卡洛树搜索

1 引言

A、引言与动机

量子计算自提出以来就表现出解决经典难解问题的强大潜力，其中一些著名的算法有区分平衡函数和常函数的 DJ 算法^[1]、量子无序搜索算法^[2]、量子大整数分解算法^[3]等。此外近些年，量子机器学习^[4]、量子哈密顿量模拟由于其相对经典算法的优越性^[5-8]都得到了广泛的研究和讨论。量子计算俨然已成为各个国家科研的新动力新方向，在 2024 年 3 月展开的十四届人大第二次会议上李强总理也提到“要开展量子技术新赛道”。

与量子算法发展的轨迹类似，近些年量子设备也得到一个长足的发展，目前量子计算设备的设计主要有几条路线：超导量子计算^[9]，离子阱量子计算^[10]等。在这些量子设备上，量子线路模型是其中用到最多的量子计算模型。量子线路是经典线路的一个推广，与经典线路不同，量子线路允许包含量子特性的逻辑门作用于量子比特。量子线路有许多不同的资源度量：其中线路规模（即量子线路所使用的逻辑门个数），线路深度（即量子线路的运行时间）以及线路使用的辅助比特个数是最受关注的量子资源，这三个资源分别对应了量子线路的最终保真度，运行时间以及空间开销。根据目前超导量子计算的发展特性，现阶段通常被称为含噪声中等规模量子计算时代 (noisy intermediate-scale quantum (NISQ) era)。在该阶段，无论是能够使用的量子比特数（空间资源），量子比特的退相干时间（时间资源）还是单个门的精度（保真度）都

还处于一个受限的阶段^[11-14]。随着量子设备的不断发展，当量子设备能够达到纠错计算的阈值时，将步入容错量子计算（fault-tolerant quantum computation era）。

在这个 NISQ 时期，量子线路设计俨然成为了连通量子算法与量子设备的桥梁。量子线路设计问题是指，对于特定的算法（通常以西矩阵形式表示），设计仅由基础量子逻辑门组成的量子线路，并使其所消耗的资源最小。量子线路设计的概念由 Barenco 等人于 1995 年提出^[15]。在这篇工作中，他们首次规范了量子基础逻辑门。此外，他们还介绍了第一个针对任意酉矩阵的通用线路设计算法，该算法所设计线路其规模和深度均为 $O(n^3 4^n)$ 。随后该结果由 Knill 等学者将线路规模和深度的上界改进到了 $O(n 4^n)$ ^[16]。紧接着 Vartiainen 等学者进一步将线路规模和深度改进到了 $O(4^n)$ ^[17]。最终在 Mottonen 等学者在保持线路规模和线路深度为 $O(4^n)$ ，且不使用辅助比特的情况下将 CNOT 门个数缩减为 $\frac{23}{48} 4^n$ ^[18]。与此同时 Shende 等学者证明了对于一般酉矩阵设计问题，存在一个实例（实际上证明可推广至对于几乎所有实例）其对应的量子线路规模至少为 $\Omega(4^n)$ ，且其对应的 CNOT 门个数至少为 $\lceil \frac{1}{4}(4^n - 3n - 1) \rceil$ ^[19]。近些年，随着对量子资源理论与量子线路设计的不断完善，量子线路资源的权衡成为更加热门的研究课题。对于深度与辅助比特之间的权衡，张等学者证明了在辅助比特充分多的情况下，对于任意 $\epsilon \leq 2^{-\Omega(n)}$ ，一个 n 比特量子状态 $|\psi_v\rangle$ 可以被一个线路深度为 $O(n^3)$ 的线路近似制备^[20]。在他们的文章中，同时提出“是否存在辅助比特和量子线路深度之间权衡”的开放性问题。随后张等学者证明，存在一个以正概率成功的概率算法，能够制备任意 n 比特量子态，其线路深度为 $O(n^2)$ ^[21]。前面两个算法所需要的辅助比特为指数规模且不可调，直到 2023 年，孙等学者证明了，对于任意 n 比特量子态，在使用 m 个辅助比特的情况下，只需要 $O(2^n/m + n \log n)$ 深度即可制备^[22]，该结果随后由袁等学者改进到 $\Theta(2^n/m + n)$ 彻底做紧了态制备问题的深度与辅助比特数的权衡^[23]。在这些资源权衡工作中，研究人员发现了对角线酉矩阵在量子线路综合问题中的基础问题，其不但能轻易转化为态制备问题，同时，一般酉矩阵也可以分解为一定数量的对角线酉矩阵。在先前的工作中^[22]，孙的结果表明，利用相位构件技术，可以高效地解决在各类不同情景下的对角线酉矩阵设计问题。此外，相位构件本身作为泡利构件的一个特例在量子化学模拟与量子哈密顿模拟中也发挥着巨大的作用。

基于此，本工作针对对角线酉矩阵问题，考虑对角线酉矩阵的设计问题。与先前的研究结果不同的是，本篇工作尝试算法保真度与线路保真度的权衡。在近几年的量子计算小规模实验中发现，通过对量子算法近似从而降低量子资源（通常会选择线路规模），最终实际线路得到的保真度反而高于对于精确算法的实现。本篇工作利用蒙特卡洛树搜索算法与相位构件技术，设计了一个针对 NISQ 时代的对角线酉矩阵的近似综合算法。该算法可以求解到任意比特数为 12 的对角线酉矩阵。当比特数为 10 时，允许误差至多为 11.96% 时，我们的算法可以节省 203 个 CNOT 门。允许误差为 40.12% 时，我们的算法可以节省 690 个 CNOT 门。

B、背景

在经典计算中，数据以具有 0 和 1 两种状态的位的形式存储。相比之下，量子计算中的数据存储于量子位中，量子位也分别由 $|0\rangle$ 和 $|1\rangle$ 表示两个基态。然而，与经典比特不同的是，量子位可以处于基态的叠加态中 $\alpha|0\rangle + \beta|1\rangle$ ，其中 α 和 β 都是复数且 $\alpha^2 + \beta^2 = 1$ 。

1、量子计算和量子门

量子比特的状态可以通过量子门来改变，量子门在数学上由酉矩阵表示。本文使用的两种重要量子门是 R_z 门、CNOT 门和 R_z 门又称相位转化门，是单量子位门，实现对该量子位的相位调整的作用。CNOT 门是双量子位门，CNOT 根据控制量子比特的状态翻转目标量子比特，也就是说通过 CNOT 实现 $|c\rangle|$

$t\rangle \rightarrow |c\rangle |c \oplus t\rangle$, 这里 $c, t \in \{0, 1\}$ 且 $\oplus$ 表示异或。量子门对应其酉矩阵表示形式：

$$R_z(\theta) = \begin{bmatrix} e^{-i\theta/2} & & \\ & e^{i\theta/2} & \\ & & 1 \end{bmatrix}, \quad CNOT = \begin{bmatrix} 1 & & & \\ & 1 & & \\ & & 1 & \\ & & & 1 \end{bmatrix}$$

其中 θ 为实数，空白位置元素为 0。

2、量子线路合成

量子计算的流行模型是线路模型，其中量子比特携带的信息被量子门修改，量子门在数学上对应于酉运算。如果一个复数方阵 U 的共轭转置 U^* 是其逆矩阵，即 $UU^* = U^*U = I$ ，则该复数方阵 U 为酉矩阵。

在线路模型中， n 个量子比特门由 $2^n \times 2^n$ 的酉矩阵 U 表示，其属于希尔伯特空间 $\mathbb{C}^{2^n}$ 。门对量子比特状态的影响是通过将酉矩阵 U 与表示量子态的向量 $|\varphi\rangle$ 相乘 $|\varphi'\rangle = U|\varphi\rangle$ 来获得的。

3、误差度量

在度量解的误差时，为了真实体现酉矩阵的差异，人们需要选择一个合适的度量指标。度量指标的选择本着相同的酉矩阵在该指标下为 0，差异越大的酉矩阵在该指标下度量值越大。在度量酉矩阵距离时，不少学者使用了原矩阵与目标矩阵的共轭转置矩阵求乘积的秩来进行衡量，如 $1 - \text{Re}(\text{Tr}(U_T^\dagger U_C))/n$ ^[24] 和 $\sqrt{1 - \|\text{Tr}(U_T^\dagger U_C)\|^2/n^2}$ ^[25]。我们认为这样可以很好的考虑到二维酉矩阵的每一个元素的差异。但是针对对角线酉矩阵，非对角线的元素全为 0，因此我们仅需考虑对角线上的元素，故我们使用二范数来度量其误差。若我们定 λ 的二范数为误差，我们有 $|\lambda\rangle = M|\alpha\rangle$ 。然而我们需要优化该目标， λ 的二范数本质为 α 与 α' 作差求内积。又因 M 为酉矩阵，因此本质为 α 和 α' 的误差，是个加性的误差，距离衡量线路差异有一定的差异。因此，我们选择 U_T 中的 $e^{i\lambda}$ 的二范数为误差，详见第三小节误差的定义及参数优化部分。

2 任务设计概述

在本节中，我们将描述解决减小对角线酉矩阵的设计的线路复杂性的问题。这里首先概述对角线酉矩阵的问题设计。

n 个量子比特上的量子变换由大小为 $2^n \times 2^n$ 的酉矩阵 U 表示。线路合成的目标是合成一个目标对角线酉矩阵 U_T 。

$$U_T = \begin{bmatrix} U_1 & & & \\ & U_2 & & \\ & & \dots & \\ & & & U_{2^n} \end{bmatrix} \quad (1)$$

其中， U_T 的参数由矩阵 $|\lambda\rangle$ 确定，表示为：

$$U_T = \begin{bmatrix} e^{i\lambda_1} & & & \\ & e^{i\lambda_2} & & \\ & & \dots & \\ & & & e^{i\lambda_{2^n}} \end{bmatrix} \quad (2)$$

参数矩阵 $|\lambda\rangle$ 由 $|\lambda\rangle = M|\alpha\rangle$ 给出。很容易理解，如果我拥有足够数量的 $CNOT$ 门，我可以选择完备的 $|\alpha\rangle$ 。但是我要使用尽量少的线路长度，达到尽量小的 $D(U_C, U_T)$ 。这里， $D(U_C, U_T)$ 为解的误差。

综合算法的质量是通过它产生的线路中的门的数量和解的误差来评估的，将在第三节中更详细地讲解了评估指标。这里我们设定了两个任务，两个任务有各自的评估目标。

A、任务一：输入 $CNOT$ 门的数量

$CNOT$ 门的数量是整个线路长度的直接指标，有了有限的 $CNOT$ 门的数量即确定了线路的长度。在确定了线路的长度后，优化目标为最小化解的误差。

B、任务二：输入解的误差

线路长度为综合算法提供了最优性准则：线路越短越好。 $CNOT$ 门的数量是整个线路长度的直接指标，因此在确定的误差的限定之下优化目标为最小化 $CNOT$ 门的数量。

3 合成算法

针对以上问题，为了在较快时间内搜索到尽量精确的解，我们设计了一个启发式算法。蒙特卡罗树搜索本是一个强化学习算法，在训练智能体在解空间的选择中有着良好的表现^[26-27]。本文使用蒙特卡罗树搜索算法，对解空间进行精细的搜索，得到一个尽量精确的解。在设计搜索树时，我们容易想到将距离某节点通过一个 $CNOT$ 门可以达到的位置设置为该节点的邻居节点。那么针对于 $Qbit$ 为 n 的问题来说，一个节点有 n 个子节点。此外，该搜索树的深度最大可以达到 2^n 。这么大规模的搜索树的时空开销是一个很大的问题。下面，我们介绍该搜索树的设计，以及我们应该如此之大的解空间采取的措施。

A、蒙特卡罗树搜索算法

我们针对于一个搜索位置设置为一个节点，又因节点通过一个 $CNOT$ 门可以达到其子节点，那么该节点及其祖先节点可以组成一个解。

对于搜索树的每一个节点，都设置六个属性，分别是 value、child、ancestor、father、visit 以及 quality。其中 value 表示对应的 α 矩阵的某个元素的位置，child 表示在 α 矩阵中通过一个 $CNOT$ 门可以达到的元素的位置，ancestor 表示该节点的祖先节点序列，其中相邻的可以通过一个 $CNOT$ 门实现，father 即为给节点的父亲节点，visit 为该节点的访问次数，quality 为该节点的最终质量。

在该搜索树算法分为选择、扩展、计算、回溯四部分，算法的完整过程如 **Aalgorithm 1** 所示。

1、选择

对于当前节点来说，依次对其每个孩子进行试探。如果该孩子节点未被访问过 ($visit=0$)，则访问该节点。若每个孩子 i 均被访问过，计算

$$w_i = \beta_1 \frac{quality_i}{visit_i} + \beta_2 \sum (n - d(i \rightarrow j)) \alpha_j [NODE_j^{visit} = 0] + \beta_3 \sqrt{\frac{visitFather_i}{visit_i}} \quad (3)$$

选择 w_i 最大的子节点作为新节点进行访问，其中， β_1 、 β_2 、 β_3 是控制参数。

(1) 通过第一部分 $\frac{quality_i}{visit_i}$ 控制，使算法下一步尽量选择当前解较好的分支进行搜索，即选择现有的较好的分支。在该部分中， $quality_i$ 表示该节点极其子孙节点所产生的解的误差之和。 $visit_i$ 表示该节点的访问次数，即为其子孙的个数加上该节点。所以，这个部分表示的是该节点极其子孙节点所产生的解的误差的平均数。可以理解为现有的解的好坏。

(2) 通过第二部分 $\sum(n - d(i \rightarrow j))\alpha_j[NODE_j^{visit} = 0]$ 控制, 使算法下一步尽量选择未来解可能较好的分支进行搜索。这一部分中, $(n - d(i \rightarrow j))$ 表示 Qbit 数量减去 $NODE_i$ 与 $NODE_j$ 的汉明距离, 表示距离越小越好。 α_j 表示在 α 矩阵中该位置参数值。若该位置未曾在祖先节点中出现过, 则 $[NODE_j^{visit} = 0]$ 值为 1, 反之为 0。由于值越大的元素对于最终计算生成的 λ 矩阵的贡献越大, 该部分指标即为趋近于值越大的 α 元素。

(3) 设置第三部分 $\sqrt{\frac{visitFather_i}{visit_i}}$ 控制, 使算法每次都有较大意愿访问访问次数较少的节点, 以扩大搜索范围。其中 $visitFather_i$ 为该节点的父节点的访问次数, $visit_i$ 为该节点的访问次数。

第一部分与第二部分保证算法可以在较短时间内搜索到一个较好的解。同时第三部分避免了算法较早陷入局部最优。通过归一化后, 加上权重控制, 即构成该指标。

2、扩展

对搜索树的扩展只需要寻找与当前节点汉明距离为 1 的非 0 位置序列, 去除掉与当前节点的父亲节点相同的序列后, 将剩余的所有序列全部作为当前节点的子节点加入搜索树中。

3、计算

根据当前节点给出的 α 矩阵的非零位置, 使用梯度下降算法进行拟合, 得到最终的 α 矩阵, 之后依据通过得到的 λ 矩阵计算解的误差为该节点 $quality_i = D(U_C, U_T)$ 。

4、回溯

对当前节点 i 的所有祖先节点执行操作: 令每个祖先节点的 $quality = quality + quality_i$, $visit$ 加一。针对输入为 $CNOT$ 的个数 C 的情况, 重复以上过程, 直到最终搜索树的深度等于 C 时停止。针对输入为解的误差, 重复以上过程, 实验解的误差达到目标解的误差为止。

Algorithm 1 蒙特卡洛树搜索算法

```

1: 初始化根节点  $Node$  的属性值, 且令  $visit=0$ 
2: while true do
3:   计算搜索树的高度  $h$ 
4:   if  $h \geq C$  then
5:     break
6:   end if
7:   for 每个子节点 do
8:     if 未被访问 then
9:       访问该节点
10:    else
11:      根据公式  $w_i = \beta_1 \frac{quality_i}{visit_i} + \beta_2 \sum(n - d(i \rightarrow j))\alpha_j[NODE_j^{visit} = 0] + \beta_3 \sqrt{\frac{visitFather_i}{visit_i}}$ , 选取
       $w_i$  最大的子节点进行访问
12:    end if
13:  end for
14:  扩展当前选择节点的子节点
15:  使用梯度下降算法进行拟合, 得到最终的  $\alpha$  矩阵, 计算解的误差为该节点  $quality_i = D(U_C, U_T)$ 
16:  对当前节点  $i$  的所有祖先节点执行操作: 令每个祖先节点的  $quality+ = quality_i$ ,  $visit+ = 1$ 
17: end while

```

B、搜索树剪枝及时间优化

前文提到过，不采取优化的搜索树拥有 $Qbit$ 的指数级别的节点数量。并且，每次选择从根节点开始，那么对于深度为 C 的搜索树来说，调用选择函数的时间复杂度为 $O(C * 2^n)$ 。

1、搜索树剪枝

剪枝主要依靠 $\frac{quality_i}{visit_i}$ 和 $\sum(n - d(i \rightarrow j))\alpha_j[NODE_j^{visit} = 0]$ 来实现，而 $\sqrt{\frac{visitFather_i}{visit_i}}$ 对于剪枝具有反向作用。 $\frac{quality_i}{visit_i}$ 可以剪去当前质量不好的节点， $\sum(n - d(i \rightarrow j))\alpha_j[NODE_j^{visit} = 0]$ 可以避免陷入已搜索过的节点的邻居圈子的节点和搜索 α 矩阵未来趋势差的节点。这里 $\sqrt{\frac{visitFather_i}{visit_i}}$ 不利于剪枝，但是搜索访问次数少的节点对于避免陷入局部最优解具有重要意义。

2、纵向搜索重复问题

在搜索树搜索的过程中，每次都要从根节点开始，重复的计算且对比着不同子节点的选择权重。对于深度过大的搜索树，这一部分占据了很大的时间开销，且后续节点权重的更新对于其距离过大的祖先节点的质量影响不大。这里，我们选择从当前节点的前面 l 个祖先节点出发，可以避免掉很多的重复计算开销。

3、参数优化的时间开销问题

我们使用梯度下降法进行参数优化，在结果讨论部分提到，梯度下降法进行参数优化会产生巨大的时间开销。

这里，我们采取的策略是，在搜索树的构建过程中，避免进行参数优化。而在搜索完成后进行参数优化部分。我们在搜索树构建的部分对于节点的解的质量评价采用 λ 的二范数来进行评价。完成搜索后再依据 $e^{i\lambda}$ 的二范数进行参数优化，这样可以节约巨大的时间开销。

λ 的二范数如式 (4) 所示， $e^{i\lambda}$ 的二范数依据式 (5)。依据欧拉公式 $e^{i\lambda} = \cos\lambda + i\sin\lambda$ 我们可以知道， λ 的二范数与 $e^{i\lambda}$ 的二范数在确定的自变量域内具有单调关系。我们在构建搜索树中依据 λ 的二范数优化好的节点就可以在参数优化中得到更低的误差。

$$D_\lambda(U_C, U_T) = \left(\sum_i (\lambda - \lambda')^2 \right)^{1/2} \quad (4)$$

C、误差的定义及参数优化

1、误差度量

合成的目标是生成一个线路，实现一个酉矩阵 U 。我们记合成的线路为 U_C ，目标酉矩阵记为 U_T 。我们的误差度量记为 $D(U_C, U_T)$ ，使用二范数表示其误差，如下式所示：

$$D(U_C, U_T) = \left(\sum_i \left(\frac{e^{i\lambda} - e^{i\lambda'}}{2} \right)^2 \right)^{1/2} \quad (5)$$

当 $D(U_C, U_T)$ 为零时，表示所合成的酉矩阵等同于目标酉矩阵。当 $D(U_C, U_T)$ 为 1 时，表示的解的误差最大。

2、 α 参数优化

(1) 初始化

设 $L = M\alpha$ ，令 α' 的初始非零值为 α 对应位置的值，其解中包含位置元素为非零，其余为零。随机初始化 α' 中非零元素的值。

(2) 计算梯度

计算误差函数 L 对 α' 的梯度:

$$\frac{\partial D(U_C, U_T)}{\partial \alpha'} = \sum_i \left(-\frac{(e^{i\lambda} - e^{i\lambda'})}{2} e^{i\lambda'} M \right) / D(U_C, U_T) \quad (6)$$

(3) 更新参数

使用梯度下降法更新 α' 的值:

$$\alpha'^{(i+1)} = \alpha'^i - \eta \frac{\partial L}{\partial \alpha'} \quad (7)$$

其中, η 是学习率。

(4) 迭代

重复步骤 (2) 和 (3), 直到误差 L 收敛, 即变化不大。最终的 α' 即为拟合的结果。

得到 α' 矩阵后, 通过

$$\lambda' = M\alpha' \quad (8)$$

计算得到 λ' , 并计算 $D(U_C, U_T)$ 。

4 结果讨论

我们设计了一个启发式算法来构建对角线酉矩阵的合成线路。在本节中, 我们将在随机对角线酉矩阵的合成上实现算法, 以说明算法的线路代价。

实验环境。我们在 Python 3.12 和 numpy 2.0.0 中实现了我们的算法。时间测试是在一台 2.3 GHz Intel(R) Core(TM) i7-12700H cpu 组成的本地笔记本电脑上运行得出。

A、针对于任务一的结果讨论

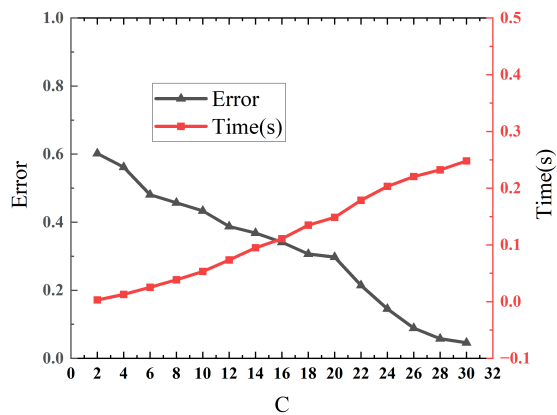
确定了已有 $CNOT$ 门的数量即为确定了线路的长度。注意, 这里针对的是随机生成的测试用例, 在不同的测试用例上会有细微的数值差异。

我们拥有较好的优化下界,依据 Fig.1, 我们可以知道, $Qbit = 5$ 时, 我们可以优化到 4.58%, 并且节省了 3 个 $CNOT$ 门。在 $Qbit = 8$ 时, 我们可以优化到 9.73%, 并且节省了 54 个 $CNOT$ 门。在 $Qbit = 10$ 时, 我们可以优化到 11.96%, 并且节省了 203 个 $CNOT$ 门。

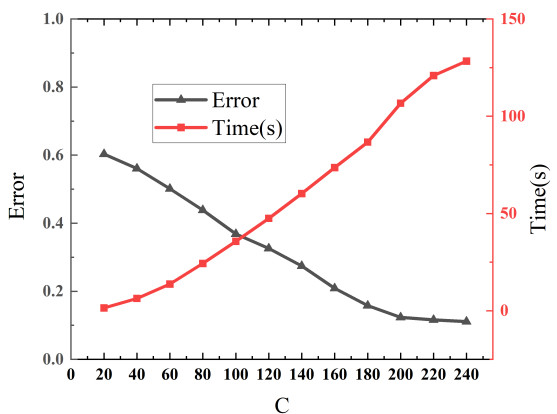
我们的算法可以节省很多 $CNOT$ 门,在进一步放松误差的时候。在 $Qbit = 8$ 时, 我们放松误差到 11.6%, 我们算法可以节省 35 个 $CNOT$ 门。当我们再放松误差到 19.52%, 我们算法可以节省 89 个 $CNOT$ 门。依据 Fig.1(c), 我们可以知道, 在 $Qbit = 10$ 时, 我们放松误差到 12.83%, 我们算法可以节省 243 个 $CNOT$ 门。当我们再放松误差到 39.93%, 我们算法可以节省 683 个 $CNOT$ 门。

此外, 我们观察下降趋势来看。我们优化算法最终在达到一定的误差时会不再下降。这里, 我们考虑可能距离为访问的节点距离较远导致的。并且, 在未达到临界误差之前, 在实验上, 误差的减少与 $CNOT$ 门的数量的增加具有近似线性关系, 这里需要后续的理论工作。

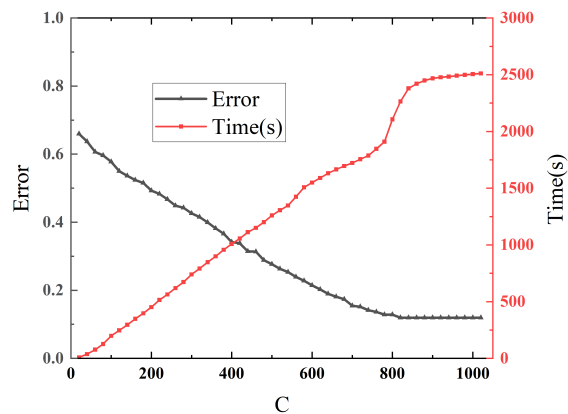
在接近优化误差临界时, 算法时间具有很大的上升, 这里说明在这些节点附近进行了巨大的搜索工作。反而误差没有得到下降, 说明达到临界状态。



(a)



(b)



(c)

Fig. 1 所示为合成解的误差 $Error$ 随着输入门的数量变化而变化。图 a 为当 $Qbit = 5$ 的情况之下，图 b 为当 $Qbit = 8$ 的情况之下，图 c 为当 $Qbit = 10$ 的情况之下。

B、针对于任务二的结果讨论

给了一个确定的误差，我们寻求用最经济的线路来实现。这里，值得注意的是，我们输入的误差是以 λ 的二范数为标准。而我们评价所用的误差为 $e^{i\lambda}$ 的二范数。这样做的原因为了节省搜索树建立中一遍又一遍的参数优化的时间开销，详细见第三部分合成算法的参数优化时间开销问题这一部分。

依据 **Fig.2**, 我们可以知道，在 $Qbit = 10$ 时，随着解与目标酉矩阵误差的减小，所需线路长度有明显的上升趋势。在 $Qbit = 10$ 时，当要求误差在误差在 40% 左右时，使用 333 个 $CNOT$ 门可以实现，节省了 690 个 $CNOT$ 门。当要求误差在误差在 20% 以下时，使用 627 个 $CNOT$ 门可以实现，节省了 396 个 $CNOT$ 门。

我们这里的工作的输入为 λ 的二范数，而误差为 U_T 中的 $e^{i\lambda}$ 的二范数。我可以确定两者之间具有单调性，但是实验上不具有线性关系。

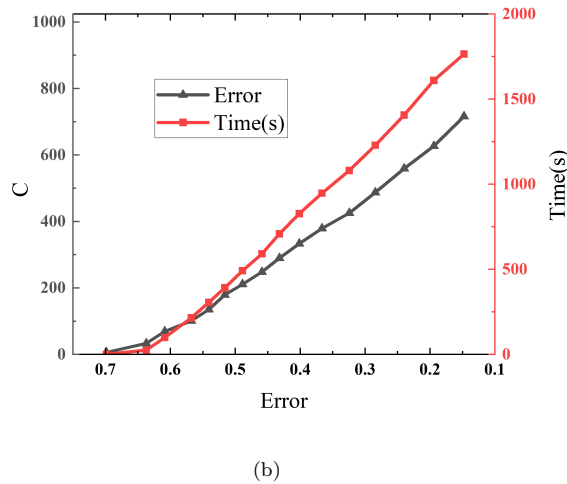
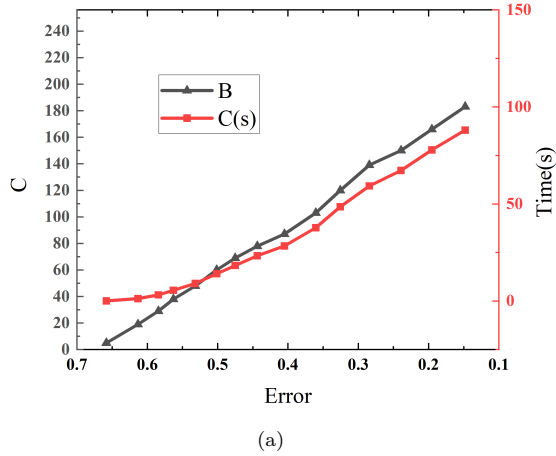


Fig. 2 所示为合成解与目标对角线酉矩阵所需输入门的数量随着误差 $Error$ 变化而变化。图 a 为当 $Qbit = 8$ 的情况之下，图 b 为当 $Qbit = 10$ 的情况之下。

C、时间开销

在普通笔记本电脑上测试我们的实验开销。针对于 $Qbit < 8$ 的情形之下，我们的算法时间长度为**秒级**。针对于 $8 \leq Qbit \leq 12$ 的情形之下，我们的算法时间长度为**分钟级**。

在算法的运行过程中，随着 $Qbit$ 的增加，算法消耗时间巨大增加。分析其中原因，是在搜索树空间，矩阵运算开销上有着指数级增长。其中，尤其是 $|\lambda\rangle = M|\alpha\rangle$ 的计算和搜索树选择部分的选择权重测算所需时间较大。尤其是当 $Qbit > 12$ 的情况下， $|\lambda\rangle = M|\alpha\rangle$ 的计算与搜索树选择部分的选择权重测算所需时间巨大，是阻碍算法时间优化的重要因素。

关于蒙特卡洛树搜索的部分，如何减少搜索空间，提高搜索效率一直是值得关注的事情。在这之中，避免重复计算、选取次优解等方法起到了重要作用。此外，原本蒙特卡洛树搜索树中分支较大，我们引入了 $\sum(n - d(i \rightarrow j))\alpha_j[NODE_j^{visit} = 0]$ 促进了搜索树的剪枝与趋向于好的生长方向。在结果上与时间上都具有很大提升作用。

5 未来工作及展望

A、针对 α 矩阵中值越大的元素的贪心算法

由于 $e^{i\lambda}$ 的二范数是由 $|\lambda\rangle$ 直接决定的。在背景部分提到又 $|\lambda\rangle = M|\alpha\rangle$ ，要优化该目标时， λ 的二范数本质为 α 和 α' 的误差，是个加性的误差。可以知道在 α 矩阵中，值越大的元素对于最终计算生成的 λ 矩阵的贡献越大，因此可以首先贪心地选择 α 中前 k 大个元素作为初始解，之后再通过搜索树算法对剩余元素进行更进一步的搜索。

1、输入 $CNOT$ 门的数量

对于门的个数为 C ，比特数为 n 的问题，通过以下算法确定最终 k 值的选取，并给出初始解。在这一部分，可以利用了 TSP 变体问题中，节点的个数与总路径长度宏观上呈现单调趋势，依据总的路径长度的倍数差异来调整所选取的门的数量 k 。并且给定误差 ϵ 留给蒙特卡洛树搜索算法来搜索。

Algorithm 2

- 1: 令 $k = \frac{C}{n}$
 - 2: 取 α 中前 k 大个元素
 - 3: 计算汉明距离 $d(k)$
 - 4: **while** $C\epsilon \leq d(k) \leq C$ **do**
 - 5: $k = k * \frac{C}{d(k)}$
 - 6: 取 α 中前 k 大个元素
 - 7: 计算汉明距离 $d(k)$
 - 8: **end while**
 - 9: 输出 α 中前 k 大个元素次序
-

2、输入解的误差

对于误差为 Error，比特数为 n 的问题，通过以下算法确定最终 k 值的选取，并给出初始解。在这一部分，可以利用了本问题中，节点的个数与解的误差 $D(U_C, U_T)$ 上呈现单调趋势，依据解的误差 $D(U_C, U_T)$ 的倍数差异来调整所选取的门的数量 k 。并且给定误差 ϵ 留给蒙特卡洛树搜索算法来搜索。

Algorithm 3

```

1: 令  $k = \frac{1}{(Error \cdot Error)}$ 
2: 取  $\alpha$  中前  $k$  大个元素
3: 代入  $D(U_C, U_T)$  得到误差  $Error(k)$ 
4: while  $Error \leq Error(k) \leq Error \cdot \epsilon$  do
5:    $k = k * \frac{Error}{Error(k)}$ 
6:   取  $\alpha$  中前  $k$  大个元素
7:   代入  $D(U_C, U_T)$  得到误差  $Error(k)$ 
8: end while
9: 输出  $\alpha$  中前  $k$  大个元素以及其位置的二进制编号字符串

```

B、求解针对于不定次序的位置所需 CNOT 门的个数问题

在解决针对 α 矩阵中值越大的元素的贪心算法之前，需要先解决一个问题，即求解针对于不定次序的位置所需 $CNOT$ 门的个数问题。该问题可以转化为如下问题：给定一组位置，表示是否取 α 矩阵中该元素。要求从零位置（初始状态）开始，仅访问每个位置一次。目标是找到一条路径，使得从初始状态到目标状态的所需的 $CNOT$ 门的个数最小。这里，容易理解，两个位置间的 $CNOT$ 门的个数即为其二进制编码的汉明距离。

最终可以形式化为以下问题。设有一组位置编号的二进制字符串集合 $S = \{s_0, s_1, \dots, s_n\}$ ，其中 s_0 是全零字符串，且 s_n 是目标字符串。定义汉明距离 $d(s_i, s_j)$ 为字符串 s_i 和 s_j 之间的汉明距离。我们希望找到一个字符串序列 $P = (s_{i_0}, s_{i_1}, \dots, s_{i_n})$ ，其中 $i_0 = 0$ ，使得下列代价函数最小化：

$$C(P) = \sum_{k=0}^{n-1} d(s_{i_k}, s_{i_{k+1}}) \quad (9)$$

其中 $C(P)$ 表示路径 P 的总汉明距离。

将字符串视为节点，将汉明距离 $d(s_i, s_j)$ 视为字符串 s_i 和 s_j 之间的带权无向边。这个问题是旅行商问题（Traveling Salesman Problem, TSP）的一种变体。与 TSP 问题不同的是，其有固定的起点，但是没有固定的终点，且每个节点（即字符串）仅访问一次。这里可以使用动态规划算法来求解该问题。

C、线路结构优化问题

在我们的工作中，我们考虑了整体的线路设定问题。考虑到局部的线路排列问题，对于线路深度、算法的时间开销等指标具有重要影响，还待考虑解决，这些的工作对于构建成一个业界可用算法具有重要意义。

参考文献

- [1] DEUTSCH D, JOZSA R. Rapid solution of problems by quantum computation[J]. Proceedings of the Royal Society of London. Series A: Mathematical and Physical Sciences, 1992, 439(1907): 553-558.
- [2] GROVER L K. A fast quantum mechanical algorithm for database search[C]//Proceedings of the twenty-eighth annual ACM symposium on Theory of computing. 1996: 212-219.
- [3] SHOR P W. Polynomial time algorithms for discrete logarithms and factoring on a quantum computer[C]//Algorithmic Number Theory, First International Symposium, ANTS-I, Ithaca, NY, USA, May 6-9, 1994, Proceedings. 1994.
- [4] BIAMONTE J, WITTEK P, PANCOTTI N, et al. Quantum machine learning[J]. Nature, 2017, 549(7671): 195-202.
- [5] BERRY D W, CHILDS A M, CLEVE R, et al. Simulating hamiltonian dynamics with a truncated taylor series[J]. Physical review letters, 2015, 114(9): 090502.
- [6] LOW G H, CHUANG I L. Optimal hamiltonian simulation by quantum signal processing[J]. Physical review letters, 2017, 118(1): 010501.
- [7] G. H. Low and I. L. Chuang. Hamiltonian simulation by qubitization[J]. Quantum, 2019, 3: 163.
- [8] BERRY D W, CHILDS A M, KOTHARI R. Hamiltonian simulation with nearly optimal dependence on all parameters[C]//2015 IEEE 56th Annual Symposium on Foundations of Computer Science. IEEE, 2015: 792-809.
- [9] BARENDT R, KELLY J, MEGRANT A, et al. Superconducting quantum circuits at the surface code threshold for fault tolerance[J]. Nature, 2014, 508(7497): 500-503.
- [10] KIELPINSKI D, MONROE C, WINELAND D J. Architecture for a large-scale ion-trap quantum computer[J]. Nature, 2002, 417(6890): 709-711.
- [11] PRESKILL J. Quantum computing in the nisq era and beyond[J]. Quantum, 2018, 2: 79.
- [12] ARUTE F, ARYA K, BABBUSH R, et al. Quantum supremacy using a programmable superconducting processor [J]. Nature, 2019, 574(7779): 505-510.
- [13] DI MATTEO O, MOSCA M. Parallelizing quantum circuit synthesis[J]. Quantum Science and Technology, 2016, 1 (1): 015003.
- [14] WANG F, LUO M, LI H, et al. Improved quantum ripple-carry addition circuit[J/OL]. Science China Information Sciences, 2016, 59. DOI: 10.1007/s11432-015-5411-x.
- [15] BARENCO A, BENNETT C H, CLEVE R, et al. Elementary gates for quantum computation[J]. Physical review A, 1995, 52(5): 3457.
- [16] KNILL E. Approximation by quantum circuits[A]. 1995.
- [17] VARTIAINEN J J, MÖTTÖNEN M, SALOMAA M M. Efficient decomposition of quantum gates[J]. Physical review letters, 2004, 92(17): 177902.
- [18] MÖTTÖNEN¹ M, VARTIAINEN J J. Decompositions of general quantum gates[J]. Trends in quantum computing research, 2006: 149.

-
- [19] SHENDE V V, MARKOV I L, BULLOCK S S. Minimal universal two-qubit controlled-not-based circuits[J]. *Physical Review A*, 2004, 69(6): 062321.
- [20] ZHANG Z, WANG Q, YING M. Parallel quantum algorithm for hamiltonian simulation[J]. *Quantum*, 2024, 8: 1228.
- [21] ZHANG X M, YUNG M H, YUAN X. Low-depth quantum state preparation[J]. *Physical Review Research*, 2021, 3(4): 043200.
- [22] SUN X, TIAN G, YANG S, et al. Asymptotically optimal circuit depth for quantum state preparation and general unitary synthesis[J/OL]. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023. DOI: 10.1109/TCAD.2023.3244885.
- [23] YUAN P, ZHANG S. Optimal (controlled) quantum state preparation and improved unitary synthesis by quantum circuits with any number of ancillary qubits[J]. *Quantum*, 2023, 7: 956.
- [24] YOUNIS E, SEN K, YELICK K, et al. Qfast: Conflating search and numerical optimization for scalable quantum circuit synthesis[C]//2021 IEEE International Conference on Quantum Computing and Engineering (QCE). IEEE, 2021: 232-243.
- [25] PATEL T, YOUNIS E, IANCU C, et al. Quest: systematically approximating quantum circuits for higher output fidelity[C]//Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems. 2022: 514-528.
- [26] BROWNE C B, POWLEY E, WHITEHOUSE D, et al. A survey of monte carlo tree search methods[J]. *IEEE Transactions on Computational Intelligence and AI in games*, 2012, 4(1): 1-43.
- [27] ZHOU X, FENG Y, LI S. A monte carlo tree search framework for quantum circuit transformation[C]//Proceedings of the 39th International Conference on Computer-Aided Design. 2020: 1-7.

附录.

A、软件包使用及展示

我们将在这一部分介绍我们的软件包的使用的操作说明。

首先,需要解压附件中的"CODE.zip",打开"CODE/dist/start.exe"。这里软件启动需要等待几秒钟的时间,等软件启动之后,依次输入 $Qbit$, 选择优化方式, 输入参数, 选择参数 λ 的输入方式并输入参数。等待软件运行即可在"start.exe"的同一目录下得到线路图。其中,输入 $Qbit$ 的界面图如 Fig.3 所示,由于求解算法的时间限制,我们限定 $Qbit \in [3, 12]$ 。选择优化方式的界面图如 Fig.4 所示,这里对应着我们的任务一与任务二,分别为输入 $CNOT$ 门的数量和输入解的误差。选择参数 $CNOT$ 门的数量的界面图如 Fig.5 所示。参数 λ 的输入方式的界面图如 Fig.6 所示,由于参数较多,我们增加为方便测试,增加了随机生成的选项。

当 $Qbit = 5$ 时,我们选择输入 $CNOT$ 门的数量且数量定为 28 得到线路图如附件中"CODE/dist/线路图 _Qbit=5_gate=28.png"所示,即 Fig.7。当 $Qbit = 8$ 时,我们选择输入生成对角线酉矩阵的误差且误差定为 0.4 得到线路图如附件中"CODE/dist/线路图 _Qbit=8_gate=99.png"所示。当 $Qbit = 10$ 时,我们选择输入 $CNOT$ 门的数量且数量定为 100 得到线路图如附件中"CODE/dist/线路图 _Qbit=10_gate=100.png"所示。

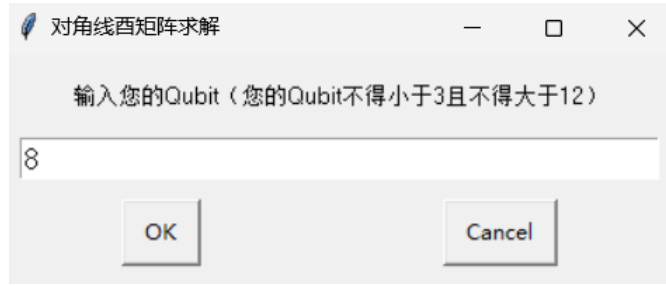


Fig. 3 输入 $Qbit$ 界面图

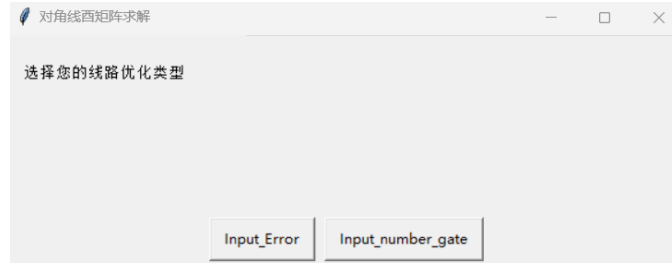


Fig. 4 选择线路优化方式界面图



Fig. 5 优化目标参数输入界面图

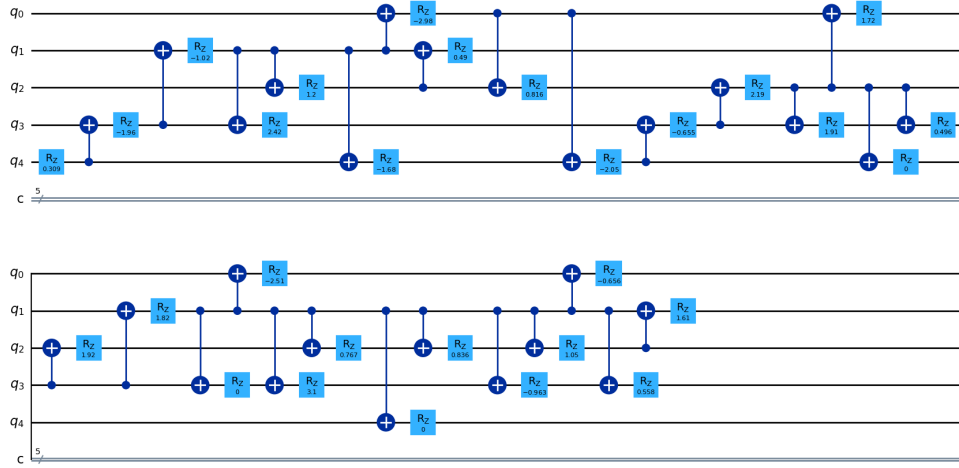
Fig. 6 参数 λ 输入方式的界面图

Fig. 7 软件输出结果线路图，当 Qbit 为 5，CNOT 门的数量为 28 时

B、软件包模块介绍及设计说明

我们设计了一款线路优化软件包,这一部分将对软件的模块构成及其设计进行说明。在软件包中,使用了 start.py、search.py、path_to_INPUTN.py、path_to_INPUTError.py、TSP.py、score.py、tool.py 和 draw.py 等部分。

1、软件启动模块

这一部分包括用户界面的可视化,由于要获取到参数较少,我们使用了简洁的 easygui 库进行了实现。通过设计的页面获取了参数之后,再调用任务实现模块进行后续工作。这一部分包含用户输入的参数 Qbit 数量 n 、 λ 矩阵、CNOT 门的数量或误差。并且初始化其他参数,酉矩阵维度 N 是通过 $N = 2^n$ 计算得来, M 矩阵通过 Walsh-Hadamard 变换得来, α 矩阵通过 $|\lambda\rangle = M|\alpha\rangle$ 计算得来。值得一提的是, M 矩阵为可逆矩阵。

2、任务实现模块

在这一部分包含任务一(输入 CNOT 门的数量)的实现和任务二(输入解的误差)实现。这一部分参数包含 Qbit 数量 n 、酉矩阵维度 N 、 λ 矩阵、 M 矩阵、 α 矩阵、CNOT 门的数量和误差。任务一与任务二的差异在与输入参数的不同(CNOT 门的数量和误差),而这个模块要保证的是问题结构的统一。

任务实现模块的主要功能为统一问题的解的数据结构。由于问题求解的蒙特卡洛树搜索部分是由树的存储结构。因此要构建节点结构。要讲初始解构建为空节点,依据问题的核心求解中的蒙特卡洛树搜索部分,将其他部分的求解的数据结构统一为节点类型。得到的最终结果再进行可视化。

3、问题求解模块

在这一部分为问题求解模块，在蒙特卡洛树搜索部分，我们设置了节点的属性，包含 `value`、`child`、`ancestor`、`father`、`visit` 以及 `quality` 这六个属性，其中 `ancestor` 即可转化成线路编译。这一部分的逻辑结构参考第三节的蒙特卡罗树搜索部分。输入 $Qbit$ 数量 n 、酉矩阵维度 N 、 λ 矩阵、 M 矩阵、 α 矩阵、初始节点、`order_index`、参数排序后的 α 矩阵和待搜索门的数量（搜索树的深度），输出即为最终结果。

4、误差计算及参数优化模块

我们将这一部分集中在了“`score.py`”，这一部分包括了蒙特卡洛树搜索的选择节点部分的前两个部分，分别为 `quality` 和 $\sum(n - d(i \rightarrow j))\alpha_j[NODE_j^{visit} = 0]$ 的计算。其中 `quality` 即为 λ 的二范数。此外，还要计算误差 $D(U_C, U_T)$ 。在计算 $D(U_C, U_T)$ 时，参数输入包括酉矩阵维度 N 、 λ 矩阵、 M 矩阵、 α 矩阵、该节点和学习率。其中，一个节点即为一个解，学习率为参数优化是使用梯度下降的学习率。

在这一部分中，误差计算依据第三部分中给出的 $D(U_C, U_T)$ ，进行计算，参数优化使用的同一小节给出的梯度下降对 α 矩阵的参数进行优化的。

5、工具函数模块

我们将这一部分集中在了“`tool.py`”，这一部分包括了计算中所频繁调用的工具函数，实现了汉明距离的计算、欧拉公式的转化、二进制转十进制、01 标准化和 α 矩阵按参数进行排序。其中，汉明距离的计算输入为两个等长 01 的字符串，输出为其汉明距离。欧拉公式的转化输入为参数 λ ，输出为其转化后的复数。

6、可视化模块

我们将这一部分集中在了“`draw.py`”，这一部分实现了可视化，其线路可视化主要依靠调用 `qiskit` 库进行实现的。这一部分的参数输入包括 $Qbit$ 数量、线路构成及偏转角度。线路构成为 01 字符串，通过 $CNOT$ 门进行实现，偏转通过 R_z 门进行实现。该模块效果输出为线路图，保存在默认目录下，即“`start.exe`”的同一目录下。